AA 274A Homework 1

Colin Skinner, *Stanford University*
colin13@stanford.edu

1. A* Motion Planning & Path Smoothing

1.1.

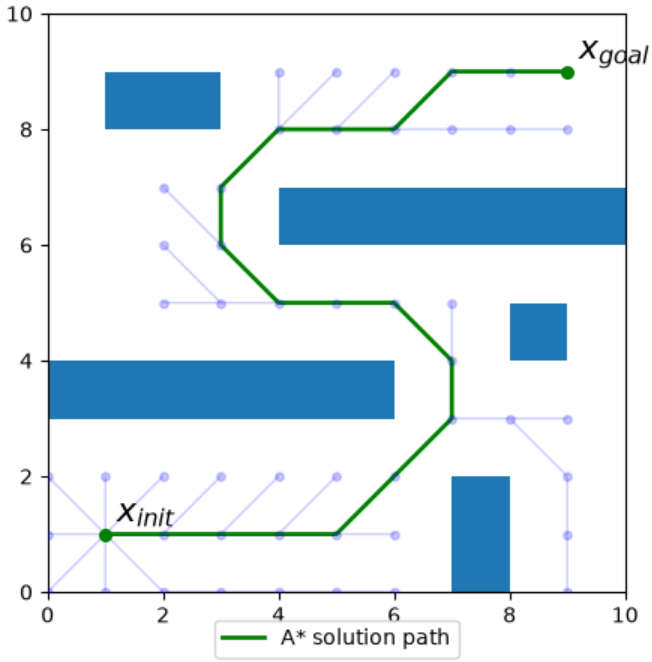


Figure 1: Basic Solution

1.2.

For the α parameter, I initially picked 0.05, but wanted to make it smoother to allow the robot to travel a shorter distance. I increased it to 0.5 and then decided to go up to the final value of 10.

When increasing the smoothing parameter α, as the trajectory becomes smoother, its length shortens. However, if it is too smooth, it might not be curved enough to avoid obstacles in an extremely cluttered environment. In this case, the obstacles are far enough from the A^* trajectory that it does not intersect no matter how much I increase it.

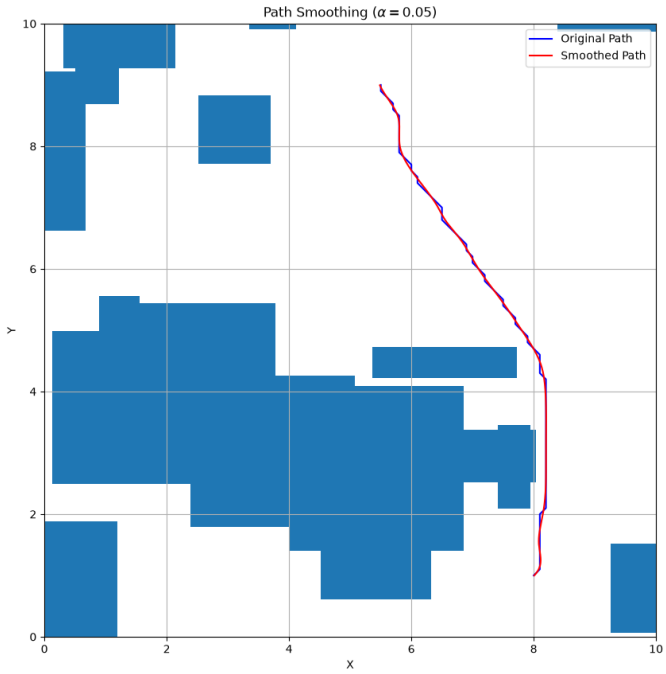


Figure 2: Trajectory with smoothing parameter of 0.05

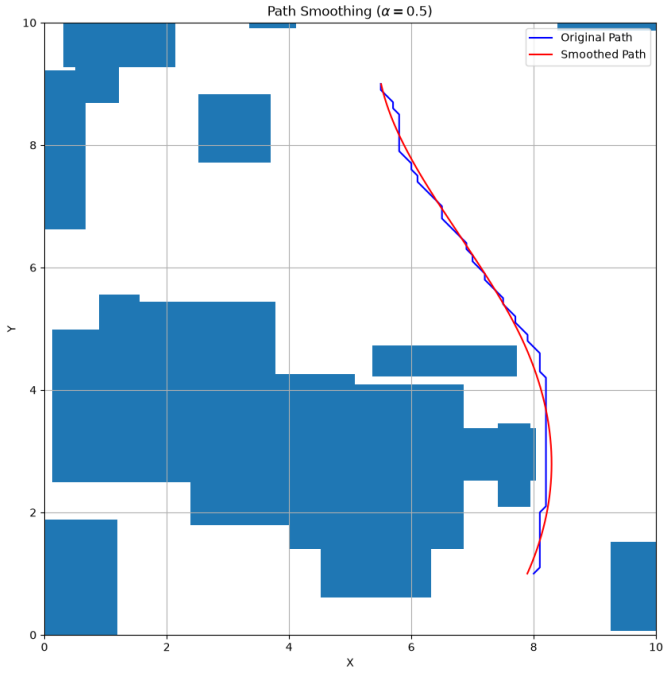


Figure 3: Trajectory with smoothing parameter of 0.5

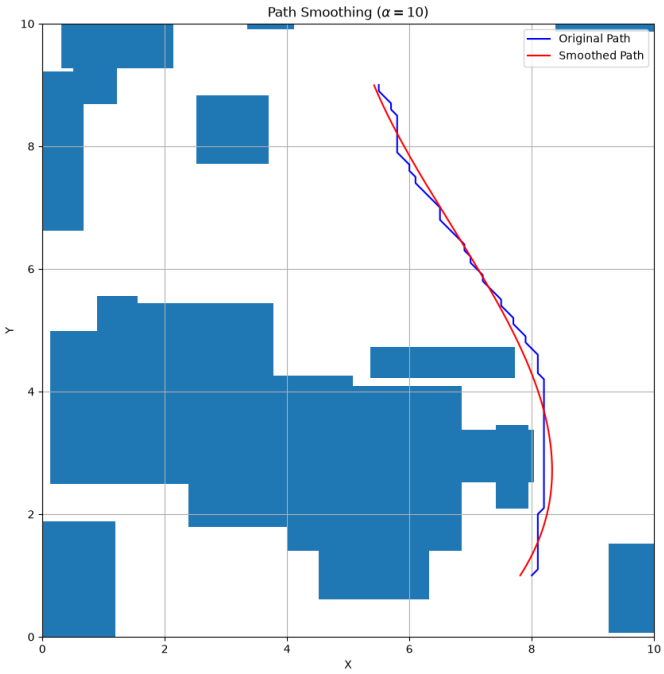


Figure 4: Trajectory with smoothing parameter of 10

1.3.

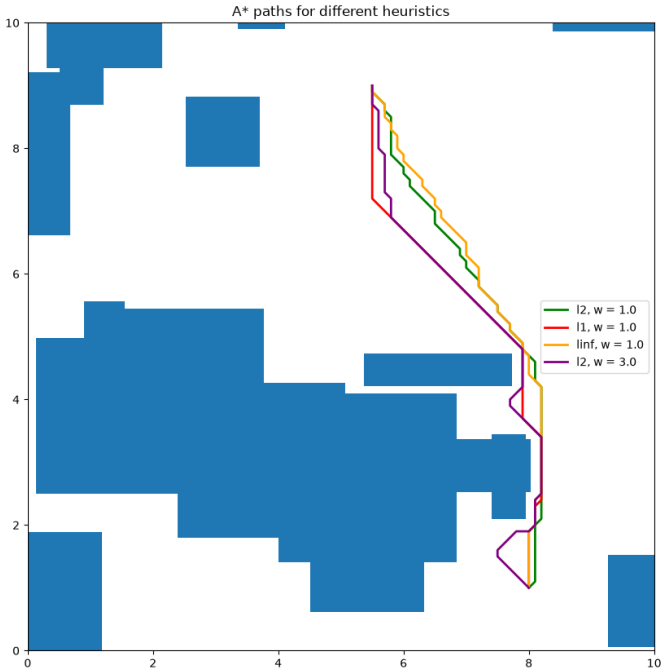


Figure 5: Weights

heuristic	w	path length	nodes expanded
12	1.0	9.201	1091

11	1.0	9.201	318
linf	1.0	9.201	1945
12	3.0	9.898	146

1.3.a

For the heuristics, the $\mathcal{L}^1$ trajectory is the only one that is not admissible. Since it totals the horizontal *and* vertical distances to the goal, if the robot is at any state that is not directly horizontal and vertical then the heuristic will overestimate the cost-to-go. For example, even in a state of $(6, 1)$, an optimal distance to the goal would have 5 straight lines and 1 diagonal to total 6.41, but the heuristic would add 6 and 1 to get 7. If, instead, diagonal moves were not possible, then it would be admissible since at most, it would be exactly equal to an optimal trajectory directly towards the goal.

For $\mathcal{L}^2$, this is the Euclidean distance, and this always takes the exact short path to the goal, so it always is equal to or less than the optimal trajectory. In the example above, the heuristic estimates 6.08, which is less than the optimal trajectory given the 8 possible moves at each state.

For $\mathcal{L}^\infty$, this takes the largest value out of both the horizontal and vertical directions. It is always smaller than a trajectory to the goal that does not lie directly horizontal or vertical to the goal, and equal when it does sit on those axes. This heuristic calculates 6 for the $(6, 1)$ state listed above.

1.3.b

In the order of nodes expanded: $\mathcal{L}^1 < \mathcal{L}^2 < \mathcal{L}^\infty$. If a heuristic is larger, then it is closer to the "true" cost and will tend to go in the direction of the optimal path. Therefore, it tends to reach the goal in relatively fewer nodes than a less informative heuristic.

The $\mathcal{L}^\infty$ norm underestimates the cost, and the $\mathcal{L}^1$ norm overestimates the cost, so both of those will explore or favor nodes that may not be close to the optimal direction (and may weigh them higher as well).

1.3.c

When the weighting is increased for the heuristic, the algorithm will tend to favor nodes that are more favored by the heuristic. In this case, sometimes the purple path ($w = 3$) will tend to drift off to the left side of the optimal path. As soon as it starts, it drifts off. Even though the optimal cost goes straight up, the upper left point has a smaller cost with the heuristic, so with the heuristic's increased weighting, it will favor the wrong node. This is shown in the path length, where it has a larger path length than the rest of the schemes.

For $\mathcal{L}^2$, increasing w from 1 to 3 increased the path length from 9.201 to 9.898, but decreased the number of expanded nodes from 1091 to 146. If the path length of 9.201 is the optimal one, then 9.898 is still, at most, less than 3 times that (optimality condition is satisfied)

$$9.898 \leq 3(9.201) = 27.603.$$

For $\mathcal{L}^1$ heuristic the other equation is

$$\|x\|_2 \leq \|x\|_1 \leq \sqrt{2}\|x\|_2.$$

which means it can overestimate the optimal cost-to-go by at most a factor of $\sqrt{2}$, giving a corresponding suboptimality bound. The result shows this too (easy to prove since both path lengths are the same for $\mathcal{L}^1$ and $\mathcal{L}^2$, so even though it is not admissible, it still produced the optimal path).

$$C_{\mathcal{L}^1} = 9.201 \leq \sqrt{2}(9.201) = 13.012.$$

Finally, a simple unit square demonstrates this, since the optimal path between diagonals is $\sqrt{2}$ but $\mathcal{L}^1$ has the heuristic at 2.

2. Rapidly-exploring Random Trees (RRT)

2.1.

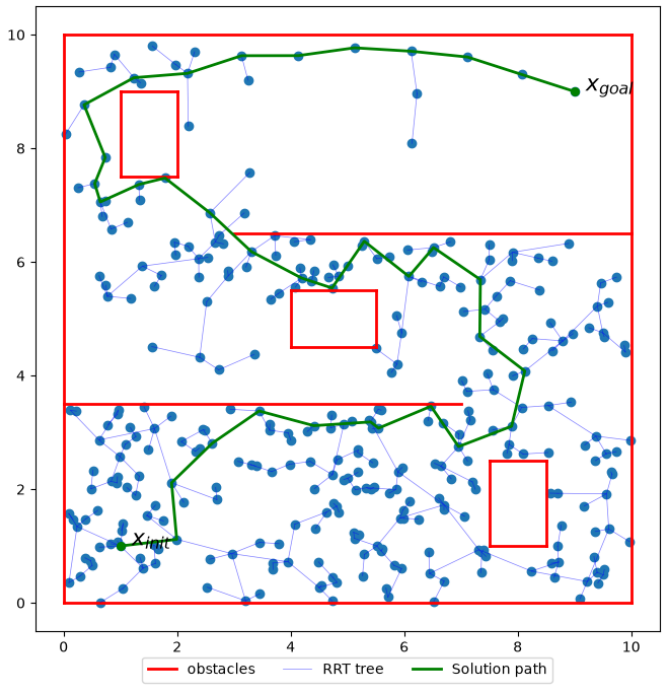


Figure 6: RRT solution with SEED $= 274$

2.2.

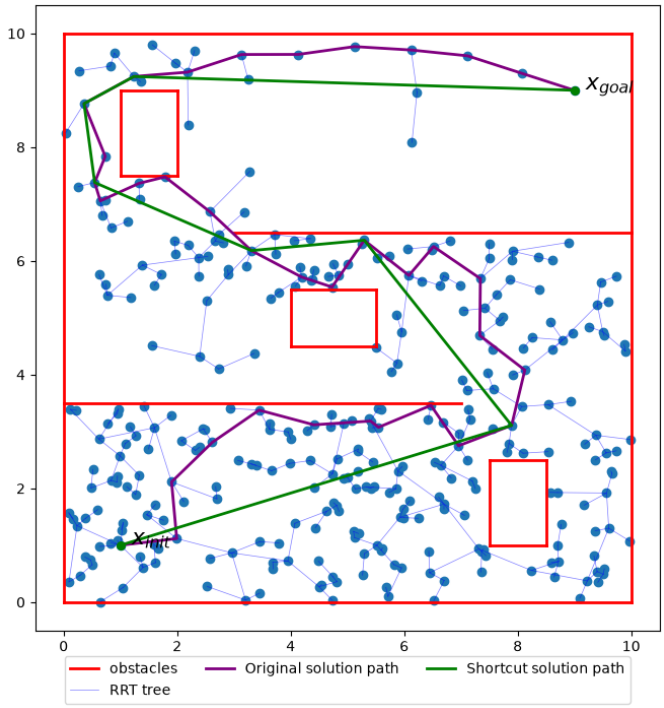


Figure 7: RRT shortcut solution with SEED $= 274$

2.3.

The table and plot for the goal-bias experiment are listed below.

goal bias	success rate	mean iterations
0.00	0.00	1999.0
0.05	1.00	390.9
0.20	1.00	403.2
0.50	1.00	614.1
0.90	0.20	1955.7

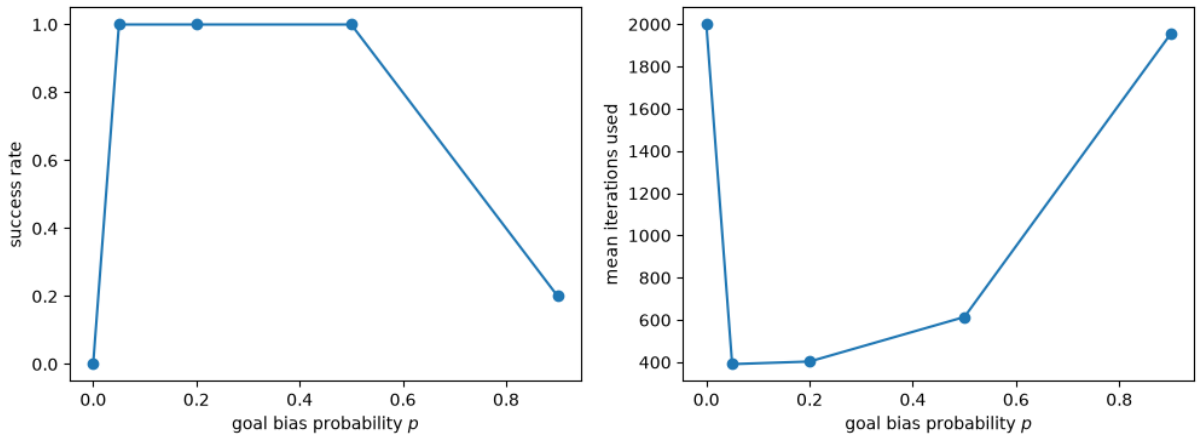


Figure 8: RRT results across varying goal biases

2.3.a

If p = 0, then the tree will never direct itself towards the goal. Even if it gets close, it still needs that small probability of having its next node be the goal in order to properly have its path go there (since it is in continuous space and not discrete space).

2.3.b

When a proposed path hits an obstacle, it just continues generating new points. If p is large, it will continually try to go towards the goal, and nearly every time, it will hit an obstacle and do nothing. When the simulation starts, the robot will hit the first wall immediately above it (after one or more steps) and then when it tries to go through, it doesn't do anything. Then it tries again, and since it has a very low chance of *not* do it (and doing a random node), it is essentially stuck or makes progress impossibly slow.

3. Heading Controller

3.1.

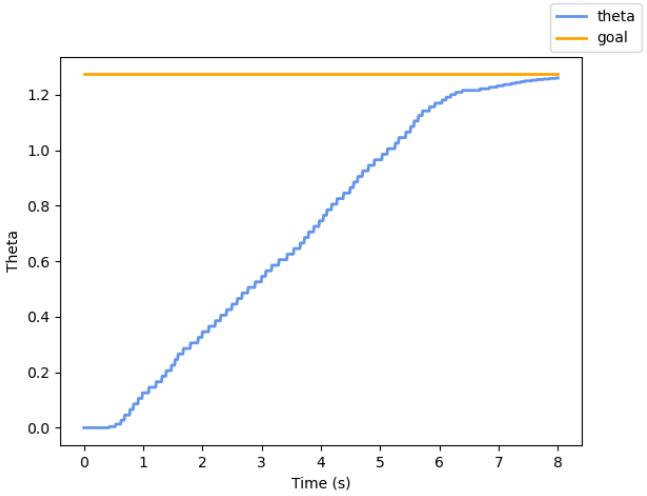


Figure 9: Commanded and goal angles

```
cskin@kevin:~/stanford/aa274a$ ros2 run autonomy_repo p3_plot.py
Commanding a goal angle, and saving a plot!
Please wait...
From initial angle of -1.7195134205394424e-12 to goal of 1.274
First 5% threshold crossing: 6.388 s
Successfully plotted to /home/cskin/stanford/aa274a/src/autonomy_repo/plots/p3_output.png
Close this node with CTRL+C!
```

Figure 10: Graph Output Text

3.2.

3.2.a

Deriving the time:

$$\begin{aligned} e(t) &= e(0)e^{-k_p t} \\ 0.05(1.274) &= 1.274e^{-2t} \\ t &= -\frac{\ln(0.05)}{2} = 1.498 \text{ s} \end{aligned}$$

Outputting the time at which the angle crosses the 5% threshold of error shows $\tau = 6.388s$.

3.2.b

For this condition to hold, we show

$$\begin{aligned} |1 - 0.1 * k_p| &< 1 \\ -1 < 1 - 0.1 * k_p &< 1 \\ -2 < -0.1 * k_p &< 0 \\ 0 < 0.1 * k_p &< 2 \end{aligned}$$

So we need this to be true for convergence

$$0 < k_p < 20$$

If $k_p < 10$, it would exponentially decay without any sign change, but $k_p > 10$ would cause the coefficient of e_k to be negative, which would cause alternating-sign error, but it would still be bound by an exponential decay envelope.

Then, if $k_p < 0$, error would just grow, and if $k_p > 20$ the error would still be sinusoidal, but it would grow exponentially as well

$$\begin{aligned} |w| &< w_{max} \\ -w_{max} < w &< w_{max} \\ -w_{max} < k_p * e &< w_{max} \\ -w_{max} < k_p * (\theta_{goal} - \theta) &< w_{max} \\ -w_{max}/k_p < \theta_{goal} - \theta &< w_{max}/k_p \\ -w_{max}/k_p < \theta_{goal} - \theta &< w_{max}/k_p \\ -w_{max}/k_p - \theta_{goal} < -theta &< w_{max}/k_p - \theta_{goal} \\ -w_{max}/k_p + \theta_{goal} < \theta &< w_{max}/k_p + \theta_{goal} \end{aligned}$$

And plugging in $w_{max} = 0.4$, $\theta_{goal} = 1.274$, and $k_p = 2$ we get:

$$1.074 < \theta < 1.474$$

So if θ is not within these bounds, the commanded ω will be larger than the threshold. Therefore, when the angle is smaller than the lower bound, the commanded angular velocity will saturate, so it will not travel as fast as it would if it were being commanded to the actual omega from the representative error.

For reasons why k_p cannot be arbitrarily large:

1. As stated in part b., it might cause the robot to overshoot or be too slow/sluggish
2. It might reach the physical limits of the motor itself. If this happens, it may try to command the motor to a higher ω than it is allowed to go, but the motor would saturate, and there would still be an error no matter how long the robot commands that ω. This could cause windup if there is an integral controller.

aa274a/hw1/P1_astar.py

```python
from __future__ import annotations

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from utils import plot_line_segments

class AStar(object):
    """Represents a motion planning problem to be solved using A*"""

    def __init__(self, statespace_lo, statespace_hi, x_init, x_goal, occupancy: DetOccupancyGrid2D, resolution=1,
                 heuristic="l2", heuristic_weight=1.0):
        self.statespace_lo = statespace_lo         # state space lower bound (e.g., [-5, -5])
        self.statespace_hi = statespace_hi         # state space upper bound (e.g., [5, 5])
        self.occupancy = occupancy                 # occupancy grid (a DetOccupancyGrid2D object)
        self.resolution = resolution               # resolution of the discretization of state space (cell/m)
        self.heuristic_type = heuristic            # norm used by the heuristic: "l1", "l2", or "linf"
        self.heuristic_weight = heuristic_weight   # weight w >= 1 on the heuristic (w > 1 gives weighted A*)
        self.x_offset = x_init
        self.x_init = self.snap_to_grid(x_init)    # initial state
        self.x_goal = self.snap_to_grid(x_goal)    # goal state

        self.closed_set = set()    # the set containing the states that have been visited
        self.open_set = set()      # the set containing the states that are candidate for future expansion

        self.est_cost_through = {}  # dictionary of the estimated cost from start to goal passing through state (often called f score)
        self.cost_to_arrive = {}    # dictionary of the cost-to-arrive at state from start (often called g score)
        self.came_from = {}         # dictionary keeping track of each state's parent to reconstruct the path

        self.open_set.add(self.x_init)
        self.cost_to_arrive[self.x_init] = 0
        self.est_cost_through[self.x_init] = self.heuristic(self.x_init)

        self.path = None        # the final path as a list of states

    def is_free(self, x):
        """
```

```
        Checks if a give state x is free, meaning it is inside the bounds of the map and
        is not inside any obstacle.
        Inputs:
            x: state tuple
        Output:
            Boolean True/False
        Hint: self.occupancy is a DetOccupancyGrid2D object, take a look at its methods for what might be
              useful here
        """
        ########## Code starts here ##########
        collision_free = self.occupancy.is_free(x)

        in_bounds = (
            x[0] >= self.statespace_lo[0] and
            x[1] >= self.statespace_lo[1] and
            x[0] < self.statespace_hi[0] and
            x[1] < self.statespace_hi[1]
        )

        return collision_free and in_bounds
        ########## Code ends here ##########

    def distance(self, x1, x2):
        """
        Computes the Euclidean distance between two states. This is the cost of
        traveling along the edge between two neighboring states, and it is used
        for the cost-to-arrive. It should NOT depend on self.heuristic_type.
        Inputs:
            x1: First state tuple
            x2: Second state tuple
        Output:
            Float Euclidean distance

        HINT: This should take one line. Tuples can be converted to numpy arrays using np.array().
        """
        ########## Code starts here ##########
        return np.linalg.norm(np.asarray(x1) - np.asarray(x2))
        ########## Code ends here ##########

    def heuristic(self, x):
        """
        Computes the weighted heuristic estimate of the cost-to-go from state x
        to self.x_goal, i.e. w * ||x - x_goal||, where w = self.heuristic_weight
        and the norm is selected by self.heuristic_type:
            "l1":   sum of absolute coordinate differences
            "l2":   Euclidean norm
            "linf": largest absolute coordinate difference
        Input:
```

```
            x: state tuple
        Output:
            Float heuristic value

        HINT: np.linalg.norm accepts an `ord` argument. Raise a ValueError for
              an unrecognized self.heuristic_type.
        """
        ########## Code starts here ##########
        if self.heuristic_type not in ["l1", "l2", "linf"]:
            raise ValueError(f"Unrecognized heuristic_type: {self.heuristic_type}")

        ord = {"l1": 1, "l2": 2, "linf": np.inf}

        return self.heuristic_weight * np.linalg.norm(np.asarray(self.x_goal) - np.asarray(x), ord=ord[self.heuristic_type])
        ########## Code ends here ##########

    def snap_to_grid(self, x):
        """ Returns the closest point on a discrete state grid
        Input:
            x: tuple state
        Output:
            A tuple that represents the closest point to x on the discrete state grid
        """
        return (
            self.resolution * round((x[0] - self.x_offset[0]) / self.resolution) + self.x_offset[0],
            self.resolution * round((x[1] - self.x_offset[1]) / self.resolution) + self.x_offset[1],
        )

    def get_neighbors(self, x):
        """
        Gets the FREE neighbor states of a given state x. Assumes a motion model
        where we can move up, down, left, right, or along the diagonals by an
        amount equal to self.resolution.
        Input:
            x: tuple state
        Ouput:
            List of neighbors that are free, as a list of TUPLES

        HINTS: Use self.is_free to check whether a given state is indeed free.
               Use self.snap_to_grid (see above) to ensure that the neighbors
               you compute are actually on the discrete grid, i.e., if you were
               to compute neighbors by adding/subtracting self.resolution from x,
               numerical errors could creep in over the course of many additions
               and cause grid point equality checks to fail. To remedy this, you
               should make sure that every neighbor is snapped to the grid as it
               is computed.
        """
        neighbors = []
```

```
        ########## Code starts here ##########

        for i in [-1,0,1]:
            for j in [-1,0,1]:
                if i == 0 and j == 0:
                    continue

                candidate = self.snap_to_grid((x[0] + i*self.resolution, x[1] + j*self.resolution))
                if not self.is_free(candidate):
                    continue

                neighbors.append(candidate)
        ########## Code ends here ##########
        return neighbors

    def find_best_est_cost_through(self):
        """
        Gets the state in open_set that has the lowest est_cost_through
        Output: A tuple, the state found in open_set that has the lowest est_cost_through
        """
        return min(self.open_set, key=lambda x: self.est_cost_through[x])

    def reconstruct_path(self):
        """
        Use the came_from map to reconstruct a path from the initial location to
        the goal location
        Output:
            A list of tuples, which is a list of the states that go from start to goal
        """
        path = [self.x_goal]
        current = path[-1]
        while current != self.x_init:
            path.append(self.came_from[current])
            current = path[-1]
        return list(reversed(path))

    def plot_path(self, fig_num=0, show_init_label=True):
        """Plots the path found in self.path and the obstacles"""
        if not self.path:
            return

        self.occupancy.plot(fig_num)

        solution_path = np.asarray(self.path)
        plt.plot(solution_path[:,0],solution_path[:,1], color="green", linewidth=2, label="A* solution path", zorder=10)
        plt.scatter([self.x_init[0], self.x_goal[0]], [self.x_init[1], self.x_goal[1]], color="green", s=30, zorder=10)
```

```
180         if show_init_label:
181             plt.annotate(r"$x_{init}$", np.array(self.x_init) + np.array([.2, .2]),
    fontsize=16)
182         plt.annotate(r"$x_{goal}$", np.array(self.x_goal) + np.array([.2, .2]),
    fontsize=16)
183         plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.03), fancybox=True,
    ncol=3)
184 
185         plt.axis([0, self.occupancy.width, 0, self.occupancy.height])
186 
187     def plot_tree(self, point_size=15):
188         plot_line_segments([(x, self.came_from[x]) for x in self.open_set if x !=
    self.x_init], linewidth=1, color="blue", alpha=0.2)
189         plot_line_segments([(x, self.came_from[x]) for x in self.closed_set if x !=
    self.x_init], linewidth=1, color="blue", alpha=0.2)
190         px = [x[0] for x in self.open_set | self.closed_set if x != self.x_init and x
    != self.x_goal]
191         py = [x[1] for x in self.open_set | self.closed_set if x != self.x_init and x
    != self.x_goal]
192         plt.scatter(px, py, color="blue", s=point_size, zorder=10, alpha=0.2)
193 
194     def solve(self):
195         """
196         Solves the planning problem using the A* search algorithm. It places
197         the solution as a list of tuples (each representing a state) that go
198         from self.x_init to self.x_goal inside the variable self.path
199         Input:
200             None
201         Output:
202             Boolean, True if a solution from x_init to x_goal was found
203 
204         HINTS:  We're representing the open and closed sets using python's built-in
205                 set() class. This allows easily adding and removing items using
206                 .add(item) and .remove(item) respectively, as well as checking for
207                 set membership efficiently using the syntax "if item in set".
208 
209                 Use self.distance for edge costs (cost-to-arrive) and
210                 self.heuristic for the estimated cost-to-go, as in Algorithm 1
211                 of the handout.
212         """
213         ########## Code starts here ##########
214 
215         # Init sets
216         self.open_set.clear()
217         self.open_set.add(self.snap_to_grid(self.x_init))
218         self.closed_set.clear()
219 
220         # Costs
221         self.cost_to_arrive[self.x_init] = 0
222         self.est_cost_through[self.x_init] = self.heuristic(self.x_init)
223 
```

```
        while len(self.open_set) > 0:
            x_curr = self.snap_to_grid(self.find_best_est_cost_through())
            if x_curr == self.x_goal:
                self.path = self.reconstruct_path()
                return True

            self.open_set.remove(x_curr)
            self.closed_set.add(x_curr)

            for x_neigh in self.get_neighbors(x_curr):
                if x_neigh in self.closed_set:
                    continue

                tentative_cost_to_arrive = self.cost_to_arrive[x_curr] +
self.distance(x_curr, x_neigh)

                if x_neigh not in self.open_set:
                    self.open_set.add(x_neigh)
                elif tentative_cost_to_arrive > self.cost_to_arrive[x_neigh]:
                    continue

                self.came_from[x_neigh] = x_curr
                self.cost_to_arrive[x_neigh] = tentative_cost_to_arrive
                self.est_cost_through[x_neigh] = tentative_cost_to_arrive +
self.heuristic(x_neigh)

        return False
        ########## Code ends here ##########

class DetOccupancyGrid2D(object):
    """
    A 2D state space grid with a set of rectangular obstacles. The grid is
    fully deterministic
    """
    def __init__(self, width, height, obstacles):
        self.width = width
        self.height = height
        self.obstacles = obstacles

    def is_free(self, x):
        """Verifies that point is not inside any obstacles by some margin"""
        for obs in self.obstacles:
            if x[0] >= obs[0][0] - self.width * .01 and \
               x[0] <= obs[1][0] + self.width * .01 and \
               x[1] >= obs[0][1] - self.height * .01 and \
               x[1] <= obs[1][1] + self.height * .01:
                return False
        return True

    def plot(self, fig_num=0):
        """Plots the space and its obstacles"""
```

```
273         fig = plt.figure(fig_num)
274         ax = fig.add_subplot(111, aspect='equal')
275         for obs in self.obstacles:
276             ax.add_patch(
277             patches.Rectangle(
278             obs[0],
279             obs[1][0]-obs[0][0],
280             obs[1][1]-obs[0][1],))
281         ax.set(xlim=(0,self.width), ylim=(0,self.height))
282
```

aa274a/hw1/P2_rrt.py

```python
import numpy as np
import matplotlib.pyplot as plt
from utils import plot_line_segments, line_line_intersection

class RRT(object):
    """ Represents a motion planning problem to be solved using the RRT algorithm"""
    def __init__(self, statespace_lo, statespace_hi, x_init, x_goal, obstacles):
        self.statespace_lo = np.array(statespace_lo)    # state space lower bound (e.g., [-5, -5])
        self.statespace_hi = np.array(statespace_hi)    # state space upper bound (e.g., [5, 5])
        self.x_init = np.array(x_init)                  # initial state
        self.x_goal = np.array(x_goal)                  # goal state
        self.obstacles = obstacles                      # obstacle set (line segments)
        self.path = None        # the final path as a list of states

    def is_free_motion(self, obstacles, x1, x2):
        """
        Subject to the robot dynamics, returns whether a point robot moving
        along the shortest path from x1 to x2 would collide with any obstacles
        (implemented as a "black box")

        Inputs:
            obstacles: list/np.array of line segments ("walls")
            x1: start state of motion
            x2: end state of motion
        Output:
            Boolean True/False
        """
        raise NotImplementedError("is_free_motion must be overriden by a subclass of RRT")

    def find_nearest(self, V, x):
        """
        Given a list of states V and a query state x, returns the index (row)
        of V such that the steering distance (subject to robot dynamics) from
        V[i] to x is minimized

        Inputs:
            V: list/np.array of states ("samples")
            x - query state
        Output:
            Integer index of nearest point in V to x
        """
        raise NotImplementedError("find_nearest must be overriden by a subclass of RRT")

    def steer_towards(self, x1, x2, eps):
```

```
        """
        Steers from x1 towards x2 along the shortest path (subject to robot
        dynamics). Returns x2 if the length of this shortest path is less than
        eps, otherwise returns the point at distance eps along the path from
        x1 to x2.

        Inputs:
            x1: start state
            x2: target state
            eps: maximum steering distance
        Output:
            State (numpy vector) resulting from bounded steering
        """
        raise NotImplementedError("steer_towards must be overriden by a subclass of RRT")

    def solve(self, eps, max_iters=1000, goal_bias=0.05, shortcut=False, plot=True):
        """
        Constructs an RRT rooted at self.x_init with the aim of producing a
        dynamically-feasible and obstacle-free trajectory from self.x_init
        to self.x_goal.

        Inputs:
            eps: maximum steering distance
            max_iters: maximum number of RRT iterations (early termination
                is possible when a feasible solution is found)
            goal_bias: probability during each iteration of setting
                x_rand = self.x_goal (instead of uniformly randly sampling
                from the state space)
            shortcut: whether to call shortcut_path on the solution (and, if
                plot is True, also plot the path before and after shortcutting)
            plot: whether to plot the tree and solution (set False for
                batch experiments)
        Output:
            Boolean, True if a solution was found. See also the
            "Intermediate Outputs" descriptions below
        """

        state_dim = len(self.x_init)

        # V stores the states that have been added to the RRT (pre-allocated at its maximum size
        # since numpy doesn't play that well with appending/extending)
        V = np.zeros((max_iters + 1, state_dim))
        V[0,:] = self.x_init    # RRT is rooted at self.x_init
        n = 1                   # the current size of the RRT (states accessible as V[range(n),:])

        # P stores the parent of each state in the RRT. P[0] = -1 since the root has no parent,
```

```
        # P[1] = 0 since the parent of the first additional state added to the RRT must have been
        # extended from the root, in general 0 <= P[i] < i for all i < n
        P = -np.ones(max_iters + 1, dtype=int)


        success = False
        self.num_iters = 0

        ## Intermediate Outputs
        # You must update and/or populate:
        #    - V, P, n: the representation of the planning tree
        #    - success: whether or not you've found a solution within max_iters RRT iterations
        #    - self.path: if success is True, then must contain list of states (tree nodes)
        #          [x_init, ..., x_goal] such that the global trajectory made by linking steering
        #          trajectories connecting the states in order is obstacle-free.
        #    - self.num_iters: the number of RRT iterations (loop passes, including ones
        #          whose extension was rejected by collision checking) that were run before
        #          returning, i.e. the iteration at which the goal was reached, or max_iters
        #          if no solution was found.

        ## Hints:
        #   - use the helper functions find_nearest, steer_towards, and is_free_motion
        #   - remember that V and P always contain max_iters elements, but only the first n
        #     are meaningful! keep this in mind when using the helper functions!
        #   - the order in which you pass in arguments to steer_towards and is_free_motion is important

        ########## Code starts here ##########

        self.path = []

        def rand_num(high, low):
            x_range = high[0] - low[0]
            y_range = high[1] - low[1]
            return np.array([np.random.random()*x_range + low[0], np.random.random()*y_range + low[1]])

        for iter in range(max_iters):
            self.num_iters = iter

            z = np.random.random()

```

```
            if z < goal_bias:
                x_rand = self.x_goal
            else:
                x_rand = rand_num(self.statespace_hi, self.statespace_lo)

            x_near_idx = self.find_nearest(V[:n], x_rand)
            x_near = V[x_near_idx]

            x_new = self.steer_towards(x_near, x_rand, eps)

            if self.is_free_motion(self.obstacles, x_near, x_new):
                V[n] = x_new
                P[n] = x_near_idx

                n += 1

                if np.allclose(x_new, self.x_goal):
                    success = True
                    break


        else:
            success = False

        idx = n-1
        while idx != -1:
            self.path.append(V[idx])
            idx = P[idx]

        self.path = np.array(self.path[::-1])


        ########## Code ends here ##########

        if not plot:
            if success and shortcut:
                self.shortcut_path()
            return success

        plt.figure()
        self.plot_problem()
        self.plot_tree(V, P, color="blue", linewidth=.5, label="RRT tree", alpha=0.5)
        if success:
            if shortcut:
                self.plot_path(color="purple", linewidth=2, label="Original solution
path")
                self.shortcut_path()
                self.plot_path(color="green", linewidth=2, label="Shortcut solution
path")
            else:
                self.plot_path(color="green", linewidth=2, label="Solution path")
```

```
180             plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.03),
    fancybox=True, ncol=3)
181             plt.scatter(V[:n,0], V[:n,1])
182         else:
183             print("Solution not found!")
184 
185         return success
186 
187     def plot_problem(self):
188         plot_line_segments(self.obstacles, color="red", linewidth=2,
    label="obstacles")
189         plt.scatter([self.x_init[0], self.x_goal[0]], [self.x_init[1],
    self.x_goal[1]], color="green", s=30, zorder=10)
190         plt.annotate(r"$x_{init}$", self.x_init[:2] + [.2, 0], fontsize=16)
191         plt.annotate(r"$x_{goal}$", self.x_goal[:2] + [.2, 0], fontsize=16)
192         plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.03), fancybox=True,
    ncol=3)
193         plt.axis('scaled')
194 
195     def shortcut_path(self):
196         """
197         Iteratively removes nodes from solution path to find a shorter path
198         which is still collision-free.
199         Input:
200             None
201         Output:
202             None, but should modify self.path
203         """
204         ########## Code starts here ##########
205 
206         success = False
207 
208         while not success:
209             success = True
210             for i in range(1, len(self.path)-1):
211                 if np.allclose(self.path[i], self.x_goal) or
    np.allclose(self.path[i], self.x_init):
212                     continue
213 
214                 if self.is_free_motion(self.obstacles, self.path[i-1],
    self.path[i+1]):
215                     self.path = np.delete(self.path, i, axis=0)
216                     success = False
217                     break
218 
219         ########## Code ends here ##########
220 
221 class GeometricRRT(RRT):
222     """
223     Represents a geometric planning problem, where the steering solution
224     between two points is a straight line (Euclidean metric)
```

```
    """

    def find_nearest(self, V, x):
        # Consult function specification in parent (RRT) class.
        ########## Code starts here ##########
        # Hint: This should take 1-3 line.
        distances = np.apply_along_axis(lambda i: np.linalg.norm((i - x)), 1, V)
        result = np.argmin(distances)

        return result
        ########## Code ends here ##########
        pass

    def steer_towards(self, x1, x2, eps):
        # Consult function specification in parent (RRT) class.
        ########## Code starts here ##########
        # Hint: This should take 1-4 line.

        diff = x2 - x1
        if np.linalg.norm(diff) < eps:
            return x2

        result = eps * diff / np.linalg.norm(diff) + x1

        return result

        ########## Code ends here ##########
        pass

    def is_free_motion(self, obstacles, x1, x2):
        motion = np.array([x1, x2])
        for line in obstacles:
            if line_line_intersection(motion, line):
                return False
        return True

    def plot_tree(self, V, P, **kwargs):
        plot_line_segments([(V[P[i],:], V[i,:]) for i in range(V.shape[0]) if P[i] >=
0], **kwargs)

    def plot_path(self, **kwargs):
        path = np.array(self.path)
        plt.plot(path[:,0], path[:,1], **kwargs)

```

A* Motion Planning

In [80]:
```
# The autoreload extension will automatically load in new code as you edit f
# so you don't need to restart the kernel every time
%load_ext autoreload
%autoreload 2
import numpy as np
import matplotlib.pyplot as plt
import scipy.interpolate
from P1_astar import DetOccupancyGrid2D, AStar
from utils import generate_planning_problem
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

In [81]:
```
# from asl_tb3_lib.navigation import TrajectoryPlan
from utils import TrajectoryPlan
```

Sanity checks

Quick checks of `distance`, `heuristic`, and `get_neighbors` (implement `is_free` and `heuristic` as well before running this cell, since the `AStar` constructor calls `heuristic`). These are not exhaustive, but every assertion should pass before you move on.

In [82]:
```
_empty = DetOccupancyGrid2D(10, 10, [])
_a = AStar((0, 0), (10, 10), (1, 1), (4, 5), _empty)
assert np.isclose(_a.distance((1, 1), (4, 5)), 5.0)

for _h, _expected in [("l1", 7.0), ("l2", 5.0), ("linf", 4.0)]:
    _a = AStar((0, 0), (10, 10), (1, 1), (4, 5), _empty, heuristic=_h)
    assert np.isclose(_a.heuristic((1, 1)), _expected), _h
    assert np.isclose(_a.distance((1, 1), (4, 5)), 5.0), "distance must not

_a = AStar((0, 0), (10, 10), (1, 1), (4, 5), _empty, heuristic="l2", heurist
assert np.isclose(_a.heuristic((1, 1)), 10.0)

assert len(_a.get_neighbors((5, 5))) == 8
assert len(_a.get_neighbors((0, 0))) == 3
print("All sanity checks passed!")
```

```
All sanity checks passed!
```

Simple Environment

(Please submit resulting plot from this section in your write-up)

Workspace

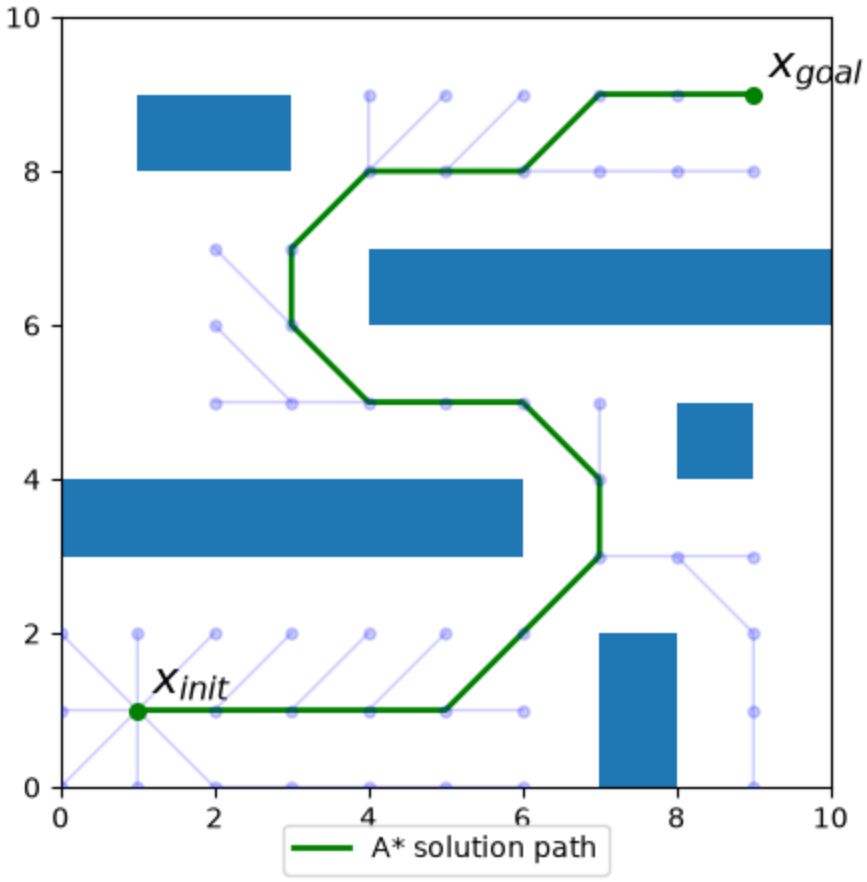
In [83]:
```
width = 10
height = 10
obstacles = [((0,3),(6,4)),((4,6),(10,7)),((7,0),(8,2)),((1,8),(3,9)),((8,4)
occupancy = DetOccupancyGrid2D(width, height, obstacles)
```

Starting and final positions

In [84]:
```
x_init = (1, 1)
x_goal = (9, 9)
```

Run A* planning

In [85]:
```
astar = AStar((0, 0), (width, height), x_init, x_goal, occupancy)
if not astar.solve():
    print("No path found")
else:
    plt.rcParams['figure.figsize'] = [5, 5]
    astar.plot_path()
    astar.plot_tree()
```



Random Cluttered Environment

Generate workspace, start and goal positions

(Try changing these and see what happens, but reset `SEED = 274` and the default parameters before generating the plots for your write-up.)

In [86]:
```
SEED = 274  # use 274 for the plots in your write-up
np.random.seed(SEED)

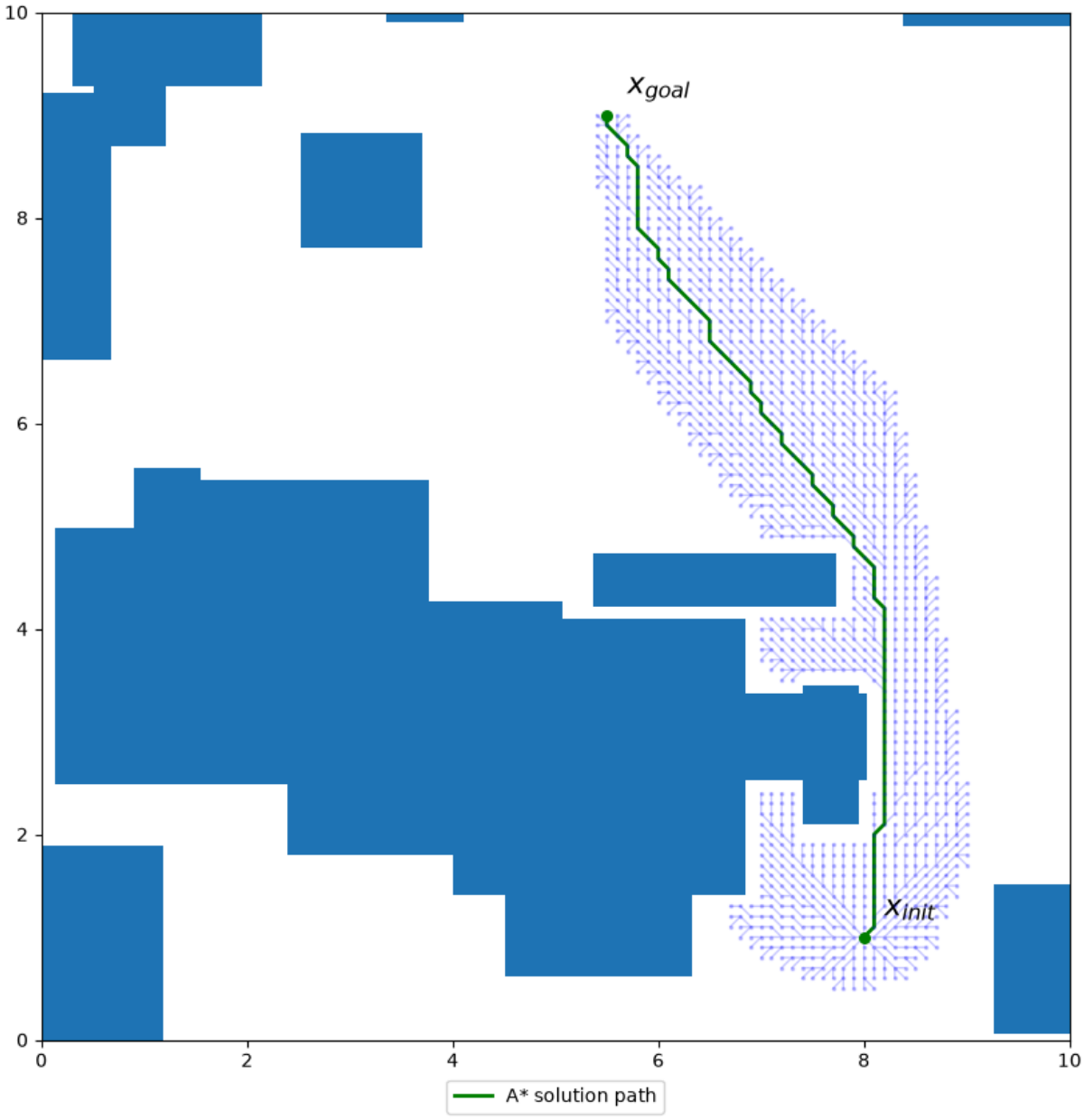
width = 10
height = 10
num_obs = 25
min_size = .5
max_size = 3

occupancy, x_init, x_goal = generate_planning_problem(width, height, num_obs
```

Run A* planning

In [87]:
```
astar = AStar((0, 0), (width, height), x_init, x_goal, occupancy, resolutior
if not astar.solve():
    print("No path found! (This is normal, try re-running the block above)")
else:
```

```
    plt.rcParams['figure.figsize'] = [10, 10]
    astar.plot_path()
    astar.plot_tree(point_size=2)
```



Smooth Trajectory

In [88]:
```
def path_length(path):
    path = np.asarray(path)
    return np.sum(np.linalg.norm(np.diff(path, axis=0), axis=1))

def compute_smooth_plan(path, v_desired=0.15, spline_alpha=0.05) -> Trajectc
    # Ensure path is a numpy array
    path = np.asarray(path)

    # Compute and set the following variables:
    #   1. ts:
```

```
    #      Compute an array of time stamps for each planned waypoint assumin
    #      velocity between waypoints.
    #
    #   2. path_x_spline, path_y_spline:
    #      Fit cubic splines to the x and y coordinates of the path separate
    #      with respect to the computed time stamp array.
    #      Hint: Use scipy.interpolate.splrep
    ##### YOUR CODE STARTS HERE #####

    dist = path_length(path)
    tspan = dist / v_desired
    n_steps = path.shape[0]
    # t = d/v
    ts = np.linspace(0, tspan, n_steps)

    path_x_spline = scipy.interpolate.splrep(ts, path[:,0], k = 3, s = splir
    path_y_spline = scipy.interpolate.splrep(ts, path[:,1], k = 3, s = splir
    ###### YOUR CODE END HERE ######

    return TrajectoryPlan(
        path=path,
        path_x_spline=path_x_spline,
        path_y_spline=path_y_spline,
        duration=ts[-1],
    )
```

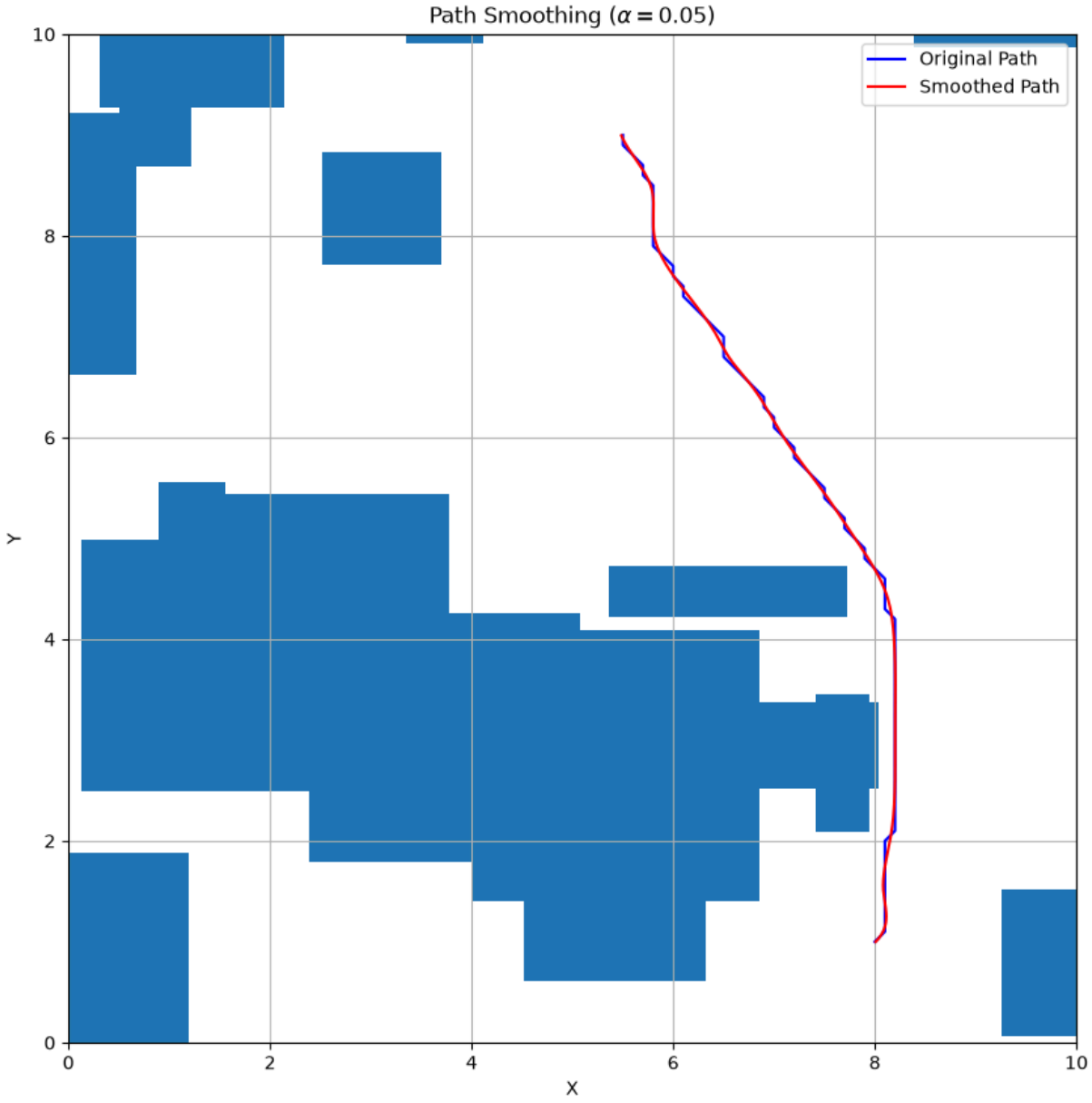
In [89]:

```
# construct a trajectory plan (0.05)
plan = compute_smooth_plan(astar.path, spline_alpha=0.05)

# plot AStar path v.s. smoothed path
astar_path = np.asarray(astar.path)
smoothed_path = plan.smoothed_path()

astar.occupancy.plot() # Added

plt.plot(astar_path[:,0], astar_path[:,1], 'b-', label='Original Path')
plt.plot(smoothed_path[:, 0], smoothed_path[:, 1], 'r-', label='Smoothed Pat
plt.legend()
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim([0, width])
plt.ylim([0, height])
plt.title(r'Path Smoothing ($\alpha = 0.05$)')
plt.grid()
```

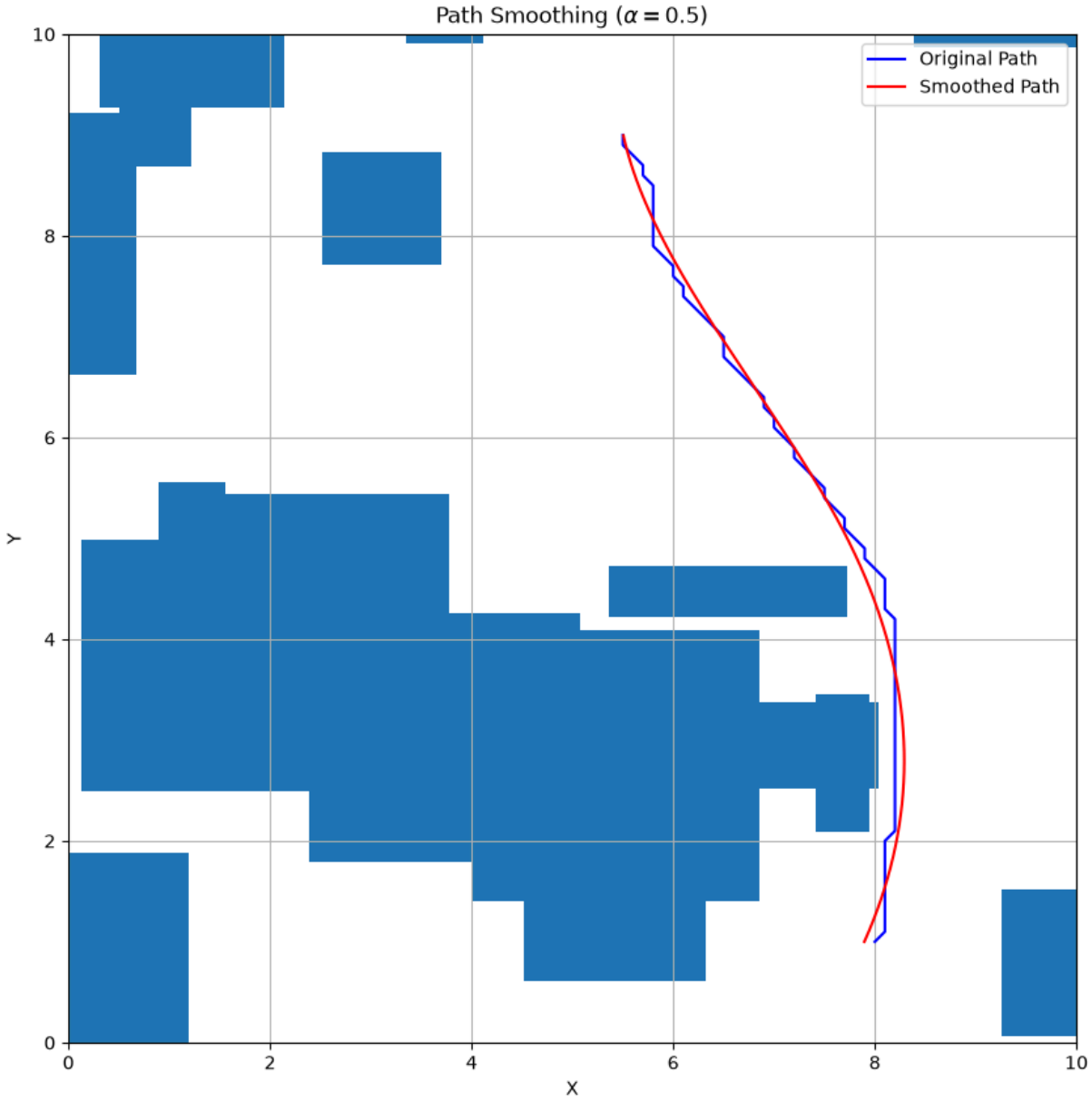


In [90]:
```
# construct a trajectory plan (0.5)
plan = compute_smooth_plan(astar.path, spline_alpha=0.5)

# plot AStar path v.s. smoothed path
astar_path = np.asarray(astar.path)
smoothed_path = plan.smoothed_path()

astar.occupancy.plot() # Added

plt.plot(astar_path[:,0], astar_path[:,1], 'b-', label='Original Path')
plt.plot(smoothed_path[:, 0], smoothed_path[:, 1], 'r-', label='Smoothed Pat
plt.legend()
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim([0, width])
plt.ylim([0, height])
plt.title(r'Path Smoothing ($\alpha = 0.5$)')
plt.grid()
```

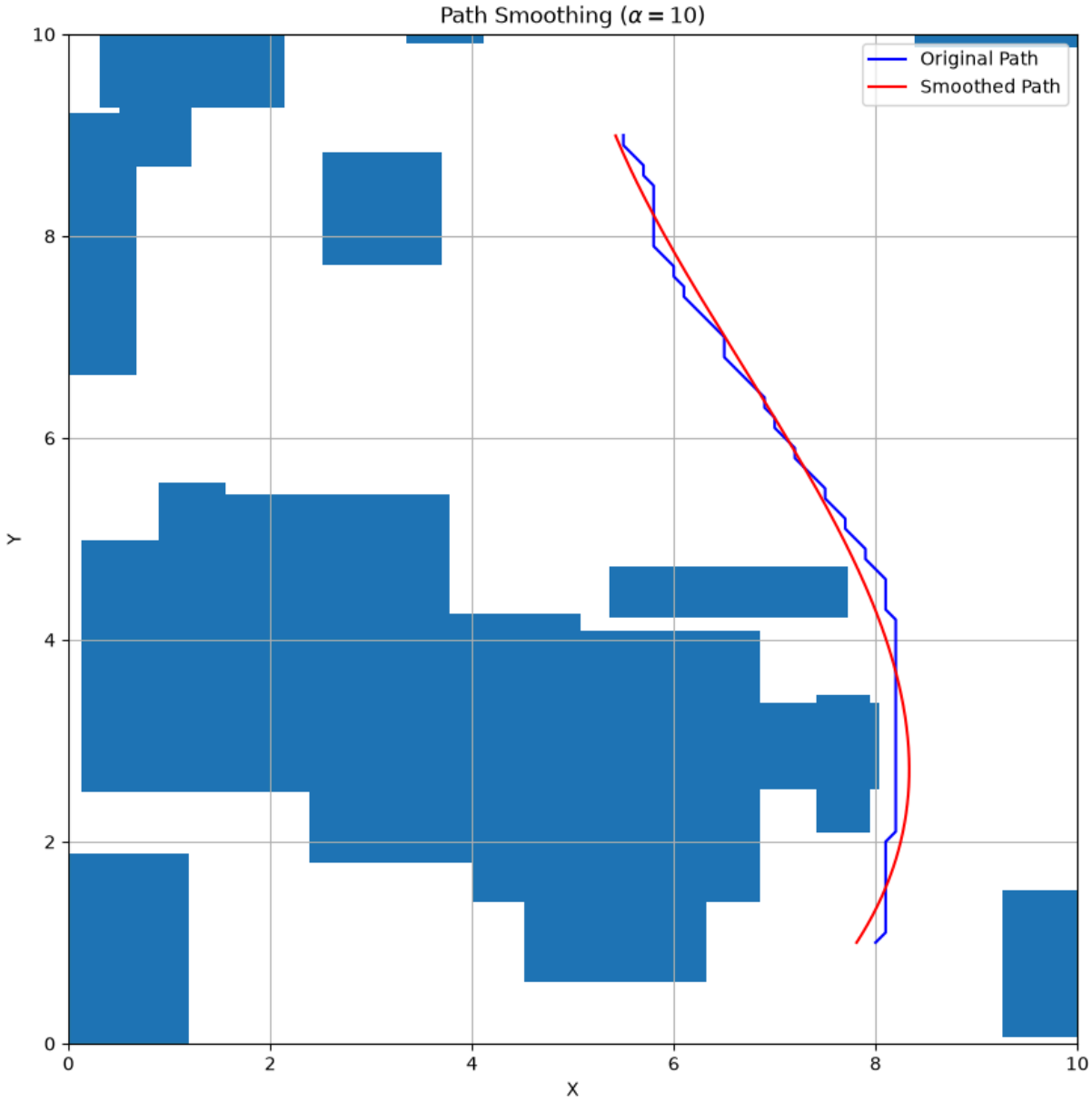


In [91]:
```
# construct a trajectory plan (10)
plan = compute_smooth_plan(astar.path, spline_alpha=10)

# plot AStar path v.s. smoothed path
astar_path = np.asarray(astar.path)
smoothed_path = plan.smoothed_path()

astar.occupancy.plot() # Added

plt.plot(astar_path[:,0], astar_path[:,1], 'b-', label='Original Path')
plt.plot(smoothed_path[:, 0], smoothed_path[:, 1], 'r-', label='Smoothed Pat
plt.legend()
plt.xlabel('X')
plt.ylabel('Y')
plt.xlim([0, width])
plt.ylim([0, height])
plt.title(r'Path Smoothing ($\alpha = 10$)')
plt.grid()
```



Heuristic Comparison

(Please submit the plot and the printed table from this section in your write-up.)

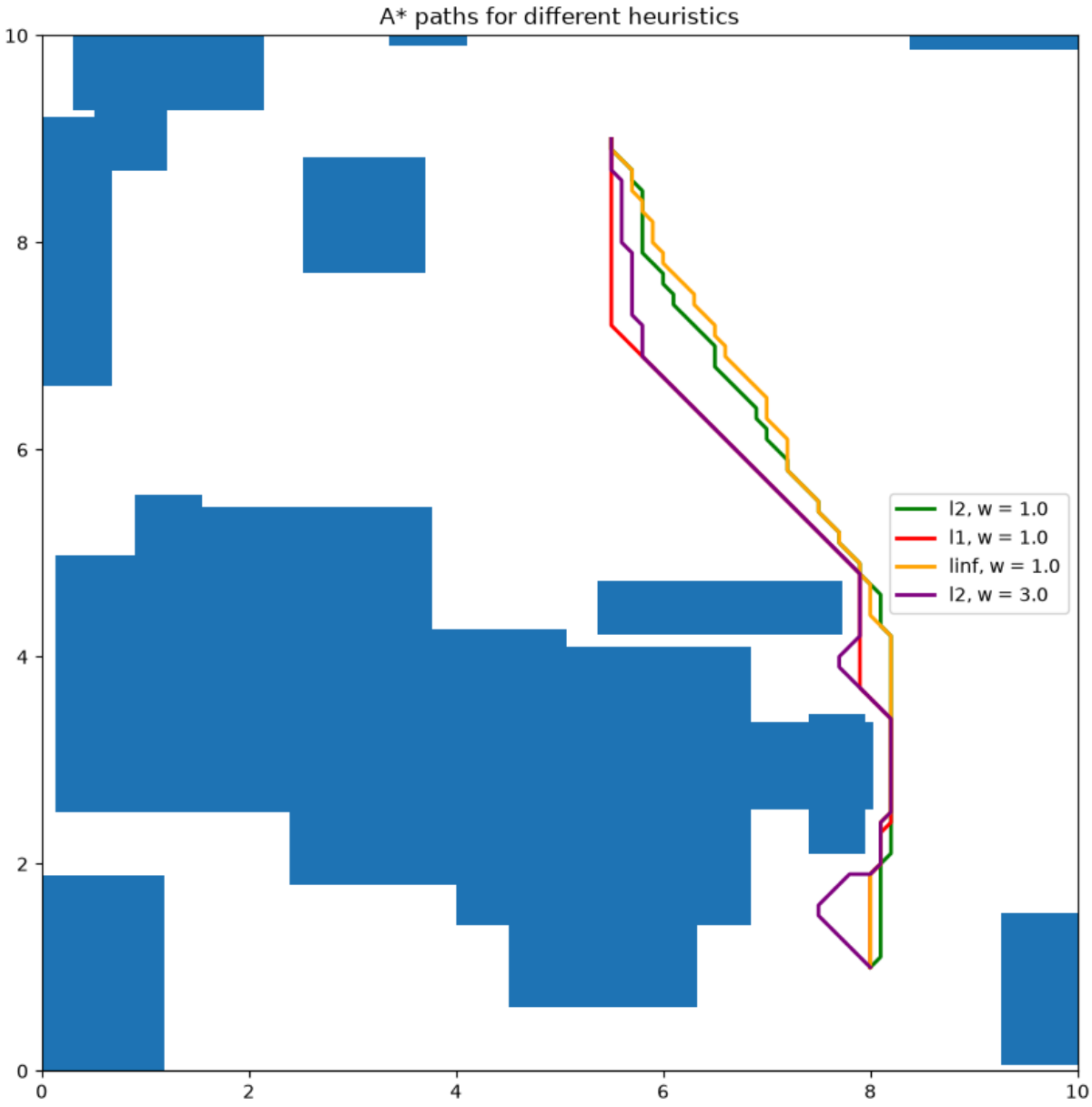
This runs A* on the random environment above (`SEED = 274`) with different heuristic norms and weights. The edge cost is always the Euclidean `distance` .

In [96]:
```
configs = [("l2", 1.0), ("l1", 1.0), ("linf", 1.0), ("l2", 3.0)]
colors = ["green", "red", "orange", "purple"]

plt.rcParams['figure.figsize'] = [10, 10]
occupancy.plot()
print(f"{'heuristic':>9} | {'w':>4} | {'path length':>11} | {'nodes expandec
print("-" * 49)
for (heuristic, weight), color in zip(configs, colors):
    astar_h = AStar((0, 0), (width, height), x_init, x_goal, occupancy, resc
```

```
                    heuristic=heuristic, heuristic_weight=weight)
    if not astar_h.solve():
        print(f"{heuristic:>9} | {weight:>4.1f} | no path found")
        continue
    print(f"{heuristic:>9} | {weight:>4.1f} | {path_length(astar_h.path):>11
    p = np.asarray(astar_h.path)
    plt.plot(p[:, 0], p[:, 1], color=color, linewidth=2, label=f"{heuristic}
plt.legend()
plt.title("A* paths for different heuristics")
plt.show()
```

```
heuristic |    w | path length | nodes expanded
-----------------------------------------------
       l2 |  1.0 |       9.201 |           1091 | 9.201219330881964
       l1 |  1.0 |       9.201 |            318 | 9.201219330881973
     linf |  1.0 |       9.201 |           1945 | 9.201219330881965
       l2 |  3.0 |       9.898 |            146 | 9.898275605729687
```



RRT Sampling-Based Motion Planning

In [ ]:
```
# The autoreload extension will automatically load in new code as you edit f
# so you don't need to restart the kernel every time
%load_ext autoreload
%autoreload 2

import numpy as np
import matplotlib.pyplot as plt
from P2_rrt import GeometricRRT
from tqdm import tqdm

plt.rcParams['figure.figsize'] = [8, 8] # Change default figure size
```

Set up workspace

In [2]:
```
def box(lo, hi):
    """Returns the four walls of an axis-aligned rectangle with corners lo a
    (x0, y0), (x1, y1) = lo, hi
    return [((x0, y0), (x1, y0)), ((x1, y0), (x1, y1)), ((x1, y1), (x0, y1))

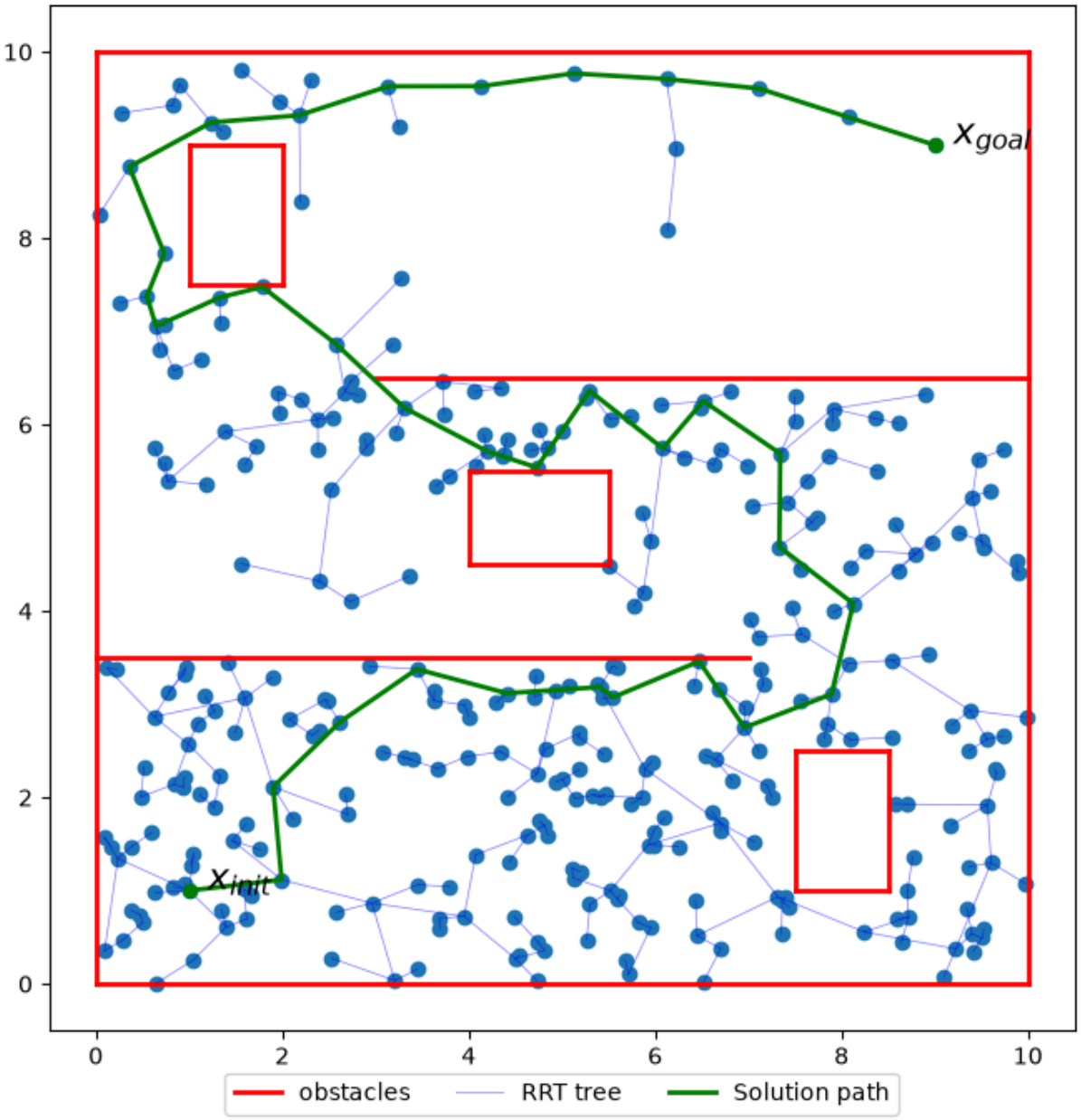
MAZE = np.array(
    box((0, 0), (10, 10))                          # outer boundary
    + [((0, 3.5), (7, 3.5)), ((3, 6.5), (10, 6.5))]  # two long walls
    + box((7.5, 1), (8.5, 2.5))
    + box((1, 7.5), (2, 9))
    + box((4, 4.5), (5.5, 5.5))
)

# try changing these!
x_init = [1, 1]  # reset to [1, 1] when saving results for submission
x_goal = [9, 9]  # reset to [9, 9] when saving results for submission
SEED = 274       # reset to 274 when saving results for submission
```

Geometric Planning

In [15]:
```
np.random.seed(SEED)
grrt = GeometricRRT([0,0], [10,10], x_init, x_goal, MAZE)
grrt.solve(1.0, 2000)
```

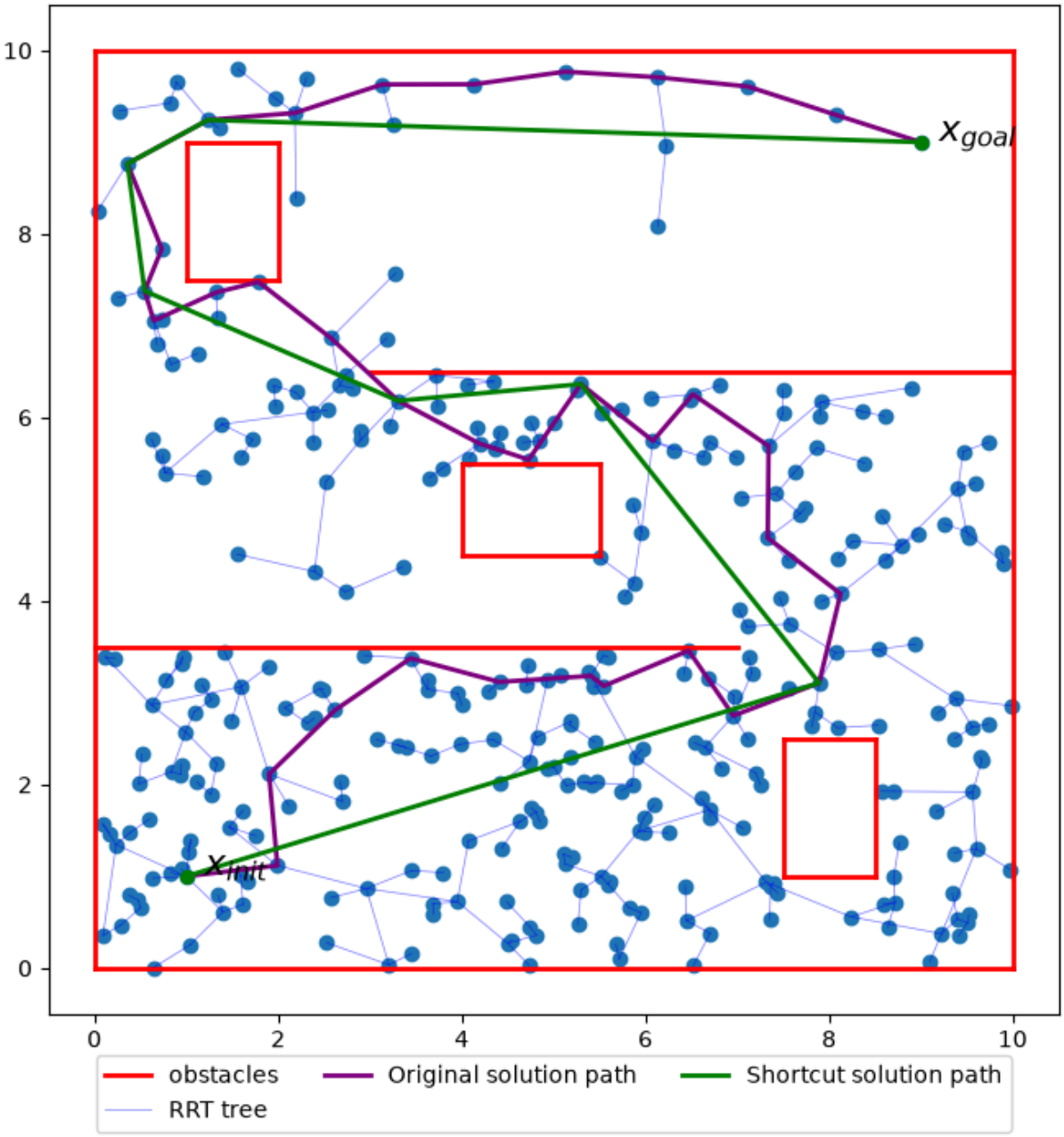
Out[15]:
```
True
```



Adding shortcutting

In [51]:
```
np.random.seed(SEED)
grrt.solve(1.0, 2000, shortcut=True)
```

Out[51]: True



Goal-bias experiment

(Please submit the plot and the printed table from this section in your write-up.)

This runs RRT (without plotting) 20 times for each goal-bias probability and records how often it succeeds and how many iterations it uses. It takes a minute or so to run.

In [52]:
```
goal_biases = [0.0, 0.05, 0.2, 0.5, 0.9]
num_trials = 20
max_iters = 2000

np.random.seed(SEED)
success_rates, mean_iters = [], []
print(f"{'goal bias':>9} | {'success rate':>12} | {'mean iterations':>15}")
print("-" * 44)
```

```
for p in goal_biases:
    iters, successes = [], 0
    for _ in range(num_trials):
        successes += grrt.solve(1.0, max_iters, goal_bias=p, plot=False)
        iters.append(grrt.num_iters)
    success_rates.append(successes / num_trials)
    mean_iters.append(np.mean(iters))
    print(f"{p:>9.2f} | {success_rates[-1]:>12.2f} | {mean_iters[-1]:>15.1f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(goal_biases, success_rates, "o-")
ax1.set_xlabel("goal bias probability $p$")
ax1.set_ylabel("success rate")
ax1.set_ylim([-0.05, 1.05])
ax2.plot(goal_biases, mean_iters, "o-")
ax2.set_xlabel("goal bias probability $p$")
ax2.set_ylabel("mean iterations used")
plt.show()
```

```
goal bias | success rate | mean iterations
---------------------------------------------
     0.00 |         0.00 |          1999.0
     0.05 |         1.00 |           390.9
     0.20 |         1.00 |           403.2
     0.50 |         1.00 |           614.1
     0.90 |         0.20 |          1955.7
```

